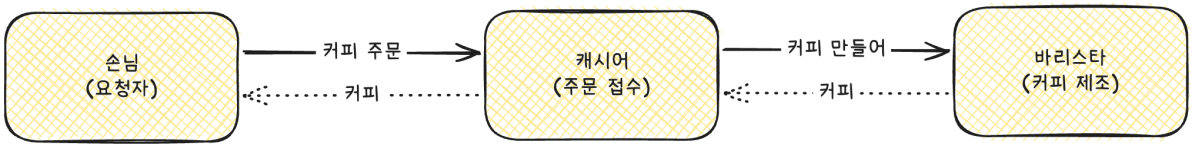
2주차

1장. 협력하는 객체들의 공동체

TL;DR 사람들은 협력하고, 협력은 역할과 책임으로 나뉘며, 이것을 그대로 객체 세계에 옮기면 객체지향이 됩니다. 그래서 객체지향은 클래스를 잘 짜는 기술이 아니라 자율적인 객체들이 협력하도록 설계하는 방식입니다.

흔히 객체지향은 현실의 사물을 소프트웨어로 옮기는 패러다임이라고 소개됩니다. 책은 이 설명이 철학적 이해에는 도움이 되지만 실무 설계에는 맞지 않는다고 말합니다. 실제 개발에서 현실의 사물과 일대일로 대응되는 객체를 찾기 어렵고, 네트워크 방화벽은 건물의 방화벽과 아무 관련이 없는데도 잘 쓰입니다. 객체지향의 목표는 현실의 모방이 아니라 새로운 세계를 만드는 것(?).

1.1. 역할, 책임, 협력



사람들은 **요청**(Request)과 **응답**(Response)을 주고받으며 **협력**(Collaboration)합니다. 이때 역할은 책임이라는 개념을 내포합니다. 선생님이라는 역할에 학생을 가르칠 책임이 따라오는 것과 같습니다. 이 개념에서 다음 네 가지 성질이 나옵니다.

- **여러 사람이 동일한 역할을 수행할 수 있다**: 손님 입장에서 커피를 받을 수만 있다면 어떤 캐시어가 처리하든 신경쓰지 않습니다.
- **역할은 대체 가능성을 의미한다**: 손님 입장에서 캐시어는 대체 가능합니다. 두 명이 동일한 역할을 수행할 수 있다면 요청자 입장에서 둘 중 어떤 사람이 역할을 수행하더라도 문제가 되지 않습니다.
- **책임을 수행하는 방법은 자율적으로 선택할 수 있다**: 바리스타가 어떻게 커피를 제조하는지는 각자에 따라 다를 수 있습니다. 이처럼 동일한 요청에 대해 서로 다른 방식으로 응답할 수 있는 능력을 다형성(Polymorphism) 이라고 합니다.
- **한 사람이 동시에 여러 역할을 수행할 수 있다**: 한 사람이 캐시어와 바리스타의 역할을 동시에 수행하는 것도 가능합니다.

책임이 설계의 품질을 좌우한다는 말을 정리하면 다음과 같습니다.

- *"책임은 객체지향 설계의 품질을 결정하는 가장 중요한 요소다."*

- *"얼마나 적절한 책임을 선택하느냐가 애플리케이션의 아름다움을 결정한다."*
- *"역할은 관련성 높은 책임의 집합이다."*

> 책임 주도 설계(Responsibility-Driven Design)
>
> 역할과 책임, 협력이라는 틀은 Rebecca Wirfs-Brock이 정립한 책임 주도 설계(Responsibility-Driven Design)에서 왔습니다. 그녀의 작업은 객체지향의 초점을 구조에서 행동과 상호작용으로 옮겨놓았고, 객체가 무엇을 책임지고 어떻게 협력하는지를 생각하게 만들었습니다. RDD 역할을 “A role is a set of related responsibilities.”라고 정의하는데, 위 인용과 정확히 같은 말입니다. 또 이 자료는 클라이언트 객체가 요청을 수행할 수 있다는 사실만 알 뿐 그 방법은 전혀 모른다는 점을 RDD의 핵심으로 짚습니다.
>
> Women in tech: Rebecca Wirfs-Brock

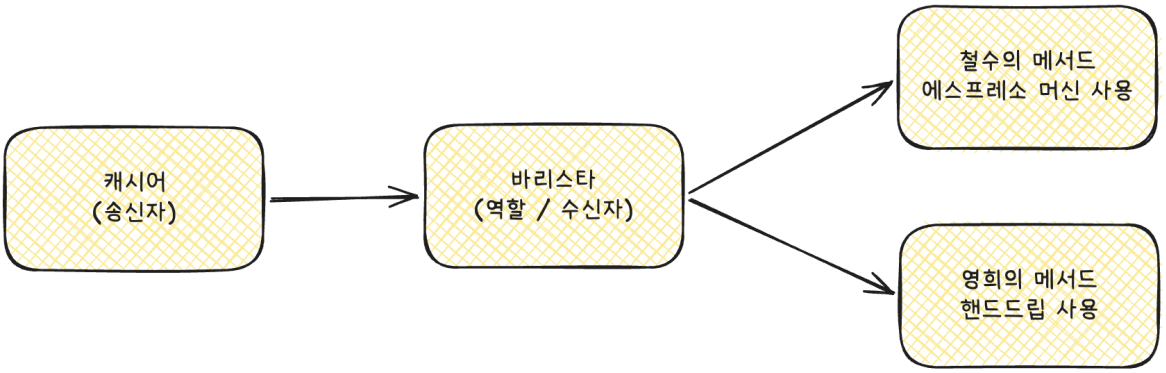
1.2. 메시지와 메서드

카페 비유를 객체지향 용어로 옮기면 이렇습니다.

카페	객체지향
사람	객체
에이전트(요청자)의 요청	메시지
요청을 처리하는 방법	메서드

협력하는 객체가 쓸 수 있는 의사소통 수단은 메시지 하나뿐입니다. 사람은 말, 글, 몸짓 등 다양한 수단으로 요청하지만, 객체는 메시지 전송과 수신만 할 수 있습니다.

아래 그림에서 보낸 쪽은 메시지만 전달하고, 받는 쪽이 메서드를 스스로 고릅니다.



> *"외부의 요청이 무엇인지를 표현하는 메시지와 요청을 처리하기 위한 구체적인 방법인 메서드를 분리하는 것은 객체의 자율성을 높이는 핵심 메커니즘이다. 이것은 캡슐화(encapsulation)라는 개념과도 깊이 관련돼 있다."*

중요한 점은 메시지와 메서드가 분리되어 있다는 것입니다. 바리스타는 "커피를 제조하라"는 메시지를 받으면 커피 머신을 쓸 수도, 손으로 내릴 수도 있습니다. 요청한 캐시어는 커피가 만들어질 것이라고 기대할 뿐 방법에는 관여하지 않습니다.

이 분리 덕분에 두 가지가 가능해집니다.

- 캡슐화: 송신자는 처리 방법을 몰라도 되고, 수신자는 방법을 숨길 수 있습니다.
- 다형성: 같은 메시지에 서로 다른 메서드가 실행될 수 있습니다.

실제 언어에서 메서드는 클래스 안의 함수로 구현되지만, 어떤 메서드가 실행될지를 실행 시점에 수신자가 선택한다는 점이 중요합니다. 프로시저 호출의 실행 코드를 컴파일 시점에 정하는 절차적 언어와 구분되는 핵심 특징입니다.

1.3. 객체의 두 가지 덕목: 협력적이고 자율적일 것

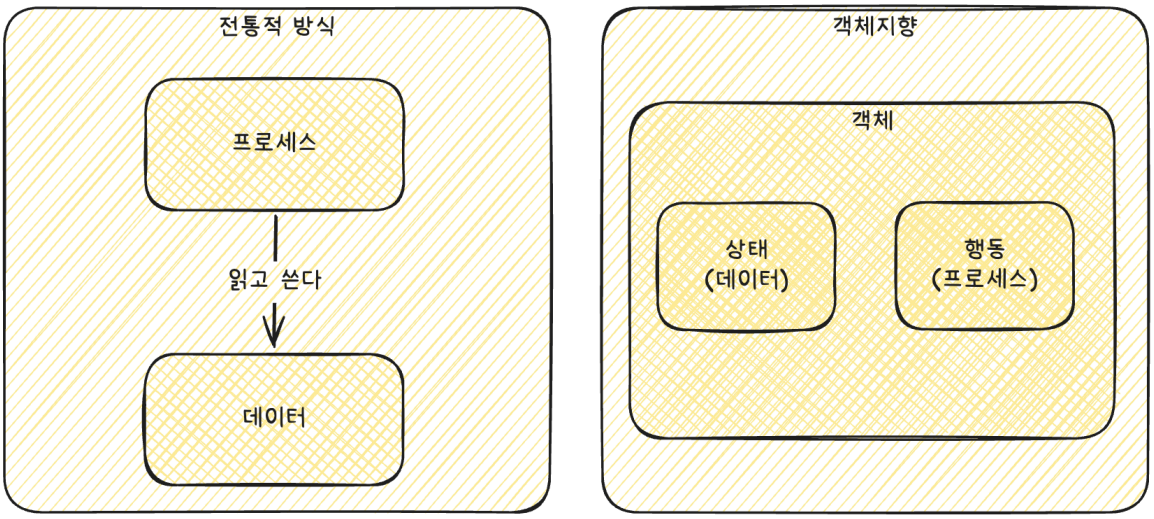
협력 공동체의 일원인 객체에게는 서로 균형을 이루어야 하는 두 가지 덕목이 있습니다.

- **협력적**: 다른 객체의 요청에 귀 기울이고, 필요하면 적극적으로 도움을 요청해야 합니다. 모든 것을 혼자 처리하려는 전지전능한 객체(god object)는 내부 복잡도 때문에 자멸합니다.
- **자율적**: 요청에 어떻게 응답할지, 심지어 응답할지 말지도 객체 스스로 판단합니다.

협력적이라는 것과 자율적이라는 것은 모순처럼 보이지만, 시키는 대로만 하는 것이 협력이 아니라 서로 요청을 주고받되 응답 방식은 각자 결정한다는 뜻으로 이해하면 됩니다.

객체가 자율적이려면 판단에 필요한 정보와 그에 따라 움직이는 능력을 함께 가져야 합니다. 바리스타가 커피 제조법을 모르면 말이 안 되는 것처럼, 객체가 어떤 행동을 한다면 그 행동에 필요한 상태도 스스로 갖고 있어야 합니다. 객체의 자율성은 내부와 외부를 명확히 나누는 데서 나옵니다. 사적인 부분은 객체가 스스로 관리하고, 외부는 허락된 통로로만 소통하며, 다른 객체가 무엇(what)을 하는지는 알아도 어떻게(how)는 알 수 없도록 구현하여야 합니다.

이 점이 전통적인 개발 방식과 객체지향을 가르는 가장 핵심적인 차이입니다.



참고: Tell, Don't Ask

데이터와 행동을 한 곳에 둔다"는 생각을 코드 수준에서 상기시켜주는 원칙이 있는데, [Tell, Don’t Ask](Tell Don't Ask입니다.
Martin Fowler는 이 원칙을 객체지향이 데이터와 그 데이터를 다루는 함수를 묶는 것이라는 점을 떠올리게 하는 도구로 설명합니다.

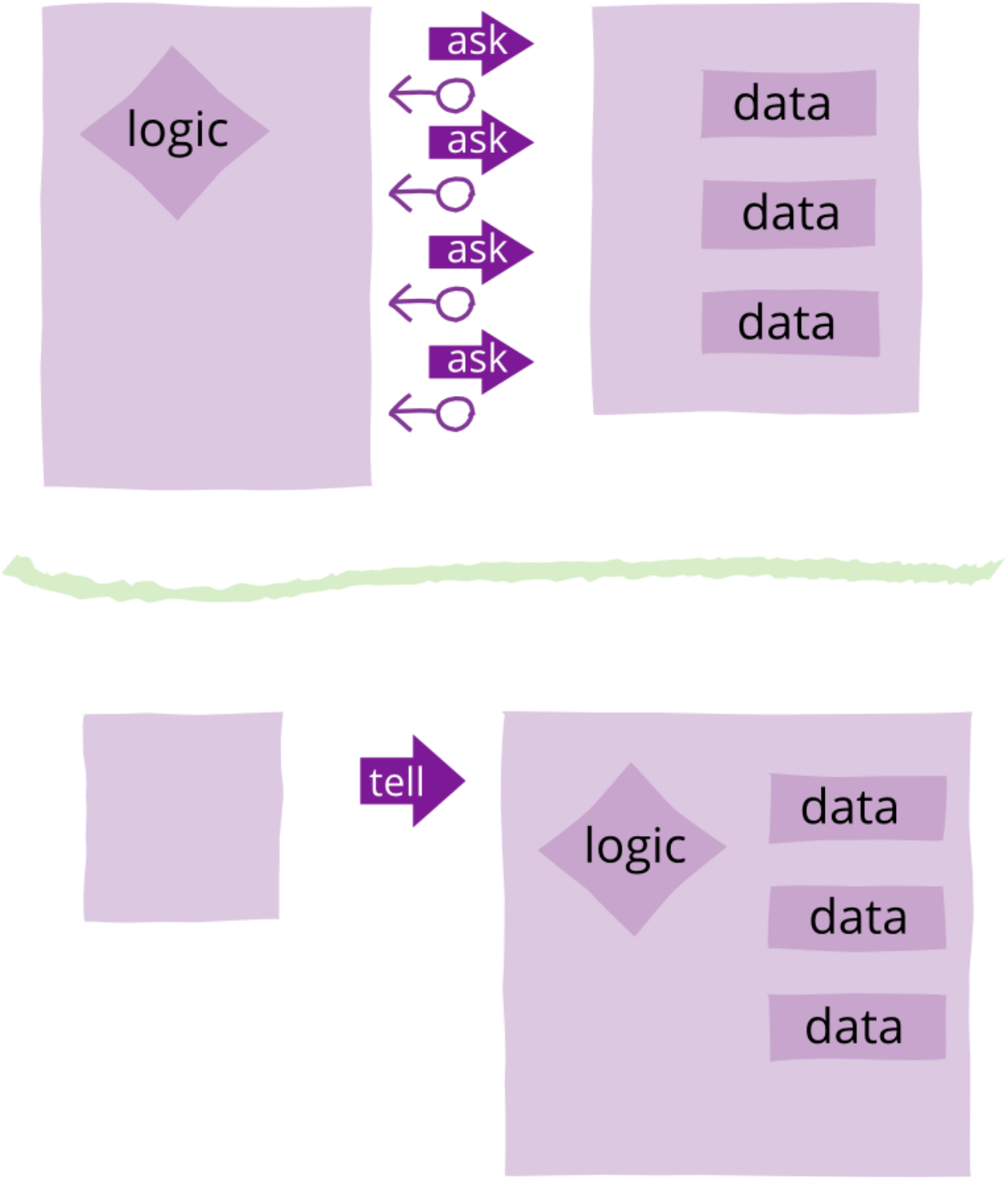
Tell과 Ask의 차이점은 아래와 같은 예시로 이해할 수 있습니다.

```
// 묻기(Ask): 상태를 꺼내 바깥에서 판단하고 변경한다
if (order.getStatus() == Status.PAID) {
    order.setStatus(Status.SHIPPED);
}

// 시키기(Tell): 요청만 하고, 판단과 변경은 객체가 자율적으로 한다
order.ship();
```

두 번째 코드에서 `order` 는 배송이 가능한 상태인지를 스스로 판단하고, 필요하면 요청을 거절할 수도 있습니다.

이를 다이어그램으로 보면 다음과 같습니다.



2장. 이상한 나라의 객체

TL;DR

2.1. 객체지향과 인지 능력

객체는 인간이 인지하고 구별할 수 있는 경계를 가진 대상이입니다.

> *"객체란 인간이 분명하게 인지하고 구별할 수 있는 물리적인 또는 개념적인 경계를 지닌 어떤 것이다."*

- 물리적인 경계: 길을 걷는 사람들, 도로 위를 지나가는 자동차
- 개념적인 경계: 오늘의 주문 내역과 어제의 주문 내역

객체지향은 소프트웨어 세계도 인간이 인지할 수 있는 객체들로 이루어져 있다는 믿음에서 출발합니다. 다만 이것을 현실을 베끼는 것으로 오해하면 안 됩니다.

> *"객체지향 패러다임의 목적은 현실 세계를 모방하는 것이 아니라 현실 세계를 기반으로 새로운 세계를 창조하는 것이다."*

현실의 전등은 사람이 켜 주어야 하지만, 소프트웨어의 전등은 스스로 켜고 끌 수 있습니다. 소프트웨어 객체는 현실 객체가 가지지 못한 능력을 가질 수 있기 때문에, 객체지향은 모방이 아니라 **창조**로 보아야 합니다.

2.2. 객체, 그리고 이상한 나라

'이상한 나라의 앨리스'에서 앨리스의 키는 계속 변합니다. 병 속의 음료를 마시면 작아지고, 케이크나 버섯을 먹으면 커집니다.

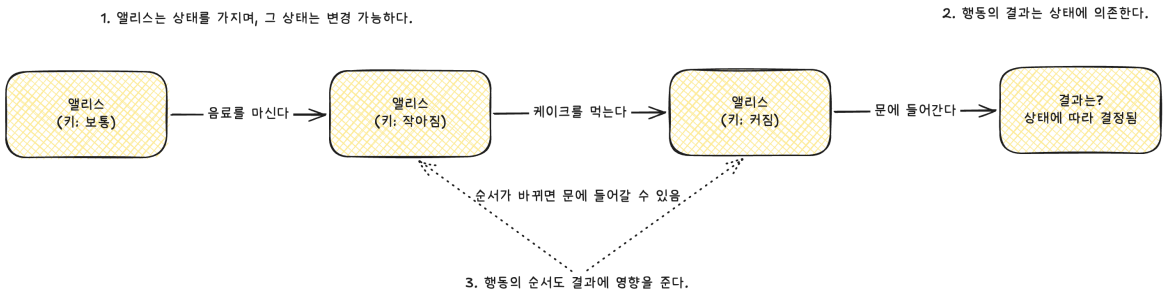
> *"결국 앨리스의 키를 변화시키는 것은 앨리스의 행동이다. 앨리스가 하는 행동에 따라 앨리스의 상태가 변하게 된다."*

> *"앨리스의 상태를 결정하는 것은 행동이지만 행동의 결과를 결정하는 것은 상태다."*

여기서 객체의 특징 세 가지가 나옵니다.

- 앨리스는 상태를 가지며 상태는 변경 가능하다.
- 앨리스의 상태를 변경시키는 것은 앨리스의 행동이다.
  - 행동의 결과는 상태에 의존적이며 상태를 이용해 서술할 수 있다.
  - 행동의 순서가 결과에 영향을 미친다.

- 앨리스는 어떤 상태에 있더라도 유일하게 식별 가능하다.



2.3. 객체, 그리고 소프트웨어 나라

> *"객체의 다양한 특성을 효과적으로 설명하기 위해서는 객체를 상태(state), 행동(behavior), 식별자(identity)를 지닌 실체로 보는 것이 가장 효과적이다."*

상태(state)

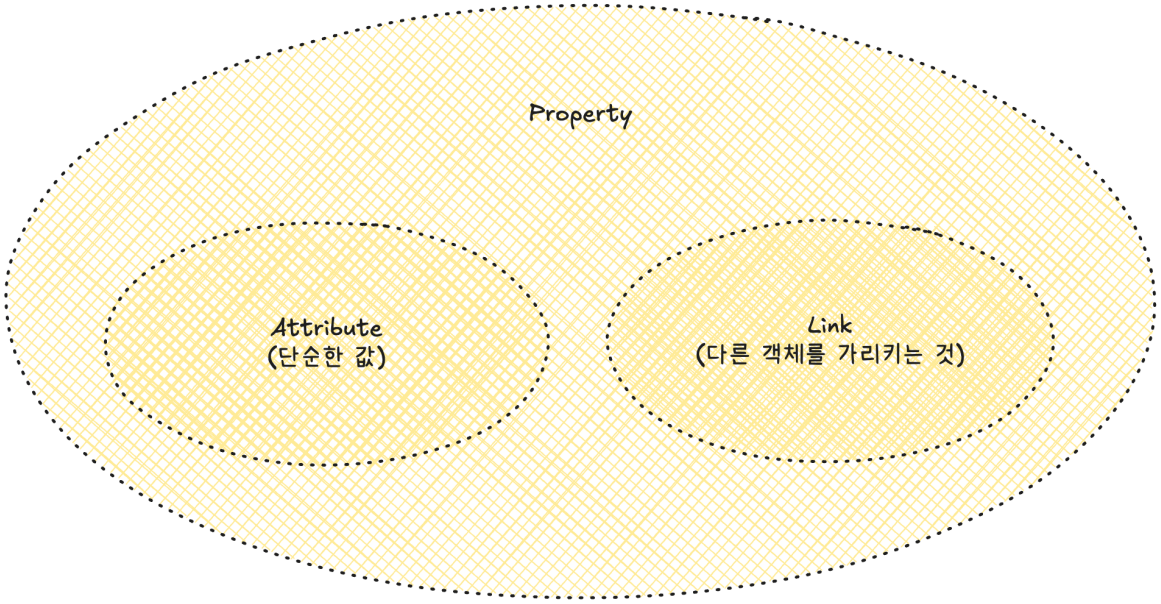
상태 없이 과거의 행동 이력을 통해서만 현재 행동의 결과를 판단하는 방식은 복잡하고 번거로우며 이해하기 어렵습니다. 예를 들어 엘리스가 문에 들어가려고 했을 때, 상태가 없다면 과거에 뭘 먹었는지 등을 모두 기억하고 있어야 하면 계산해야 합니다.

> *"상태를 이용하면 과거의 모든 행동 이력을 설명하지 않고도 행동의 결과를 쉽게 예측하고 설명할 수 있다."*

상태가 있다면 앨리스가 문을 통과할 수 있는지 알기 위해 앨리스가 과거에 무엇을 먹고 마셨는지를 추적할 필요가 없습니다. 지금의 키와 문의 높이만 알면 됩니다. 따라서 상태는 과거 이력을 현재 시점의 정보로 압축한 것으로 볼 수 있습니다.

상태는 아래와 같이 프로퍼티, 속성, 링크라는 용어로 세분화됩니다.

용어	의미	앨리스의 예
프로퍼티(property)	객체의 상태를 구성하는 모든 특징	키, 위치, 들고 있는 음료
속성(attribute)	프로퍼티 중 단순한 값	키, 위치
링크(link)	프로퍼티 중 다른 객체를 가리키는 것	들고 있는 음료(다른 객체)



행동(behavior)

객체의 상태를 바꾸는 것은 객체 자신의 행동뿐입니다. 이렇게 해야 객체의 자율성이 지켜집니다. 행동은 상태에 영향을 받고(조건에 따라 결과가 달라지고), 상태를 변경합니다. 행동에는 두 가지 부수효과가 있고, 둘 다 분명히 서술되어야 합니다.

- 객체 자신의 상태 변경
- 협력하는 다른 객체에 대한 메시지 전송

> *"행동이란 외부의 요청 또는 수신된 메시지에 응답하기 위해 동작하고 반응하는 활동이다."*

그리고 객체는 행동을 통해 다른 객체와의 협력에 참여하므로 행동은 외부에 가시적이어야 합니다.

상태 캡슐화

객체는 상태를 숨기고 행동만 외부에 노출합니다. 외부에서 객체에 접근하는 유일한 방법이 행동입니다.

> *"객체의 행동을 유발하는 것은 외부로부터 전달된 메시지지만 객체의 상태를 변경할지 여부는 객체 스스로 결정한다."*

송신자는 상태가 바뀌기를 기대할 수는 있지만, 수신자가 바꾸지 않기로 하면 간섭할 방법이 없습니다. 그래서 상태를 잘 정의된 행동 뒤로 캡슐화하면 객체의 자율성이 높아지고 협력이 단순하고 유연해집니다.

식별자

객체가 식별 가능하다는 것은 객체를 서로 구별할 수 있는 특정한 프로퍼티가 객체 안에 존재한다는 것을 의미하며, 이 프로퍼티를 식별자라고 합니다.

값과 객체의 가장 큰 차이점은 값은 식별자를 가지지 않지만 객체는 식별자를 가진다는 점입니다. 그럼 식별자가 없는 값의 경우, 어떻게 인스턴스끼리 구분할 수 있을까요?

값(value)은 숫자, 문자열, 날짜, 시간, 금액 등과 같이 변하지 않는 양을 모델링합니다. 흔히 값의 상태는 변하지 않기 때문에 불변 상태(immutable state)를 가진다고 말합니다.
값은 상태가 결코 변하지 않습니다. 따라서 어떤 시점에 동일한 타입의 두 값이 같다면 언제까지라도 두 값은 동등한 상태를 유지할 것입니다.

> *"값의 경우 두 인스턴스의 상태가 같다면 두 인스턴스를 같은 것으로 판단한다."*

이처럼 상태를 이용해 두 값이 같은지 판단할 수 있는 성질을 **동등성**(equality)이라고 합니다.

참고: 값이 왜 Immutable State를 가지는거죠?

값이 Immutable State를 가진다는 설명을 보고 처음에는 이해가 잘 되지 않았습니다. 예를 들어 아래의 예시를 보면, 값도 바뀔 수 있는 거 아닌가라는 생각을 했었습니다.

```
int value = 10;
value = 11; // 바뀐 거 아님?
```

이 오해는 **변수**와 값의 개념을 헷갈렸기 때문이었습니다.
위의 예시에서 바뀐 것은 값이 아니라 **변수**입니다. `value`는 값을 담는 이름 붙은 공간이고, 10과 11은 그 공간에 담기는 값 자체입니다. `10 == 10`, `11 == 11`, `10 != 11` 은 언제나 참이고, 10이 11로 변하는 일은 없습니다. 변수가 가리키는 값만 교체된 것입니다. 반면 객체는 같은 객체가 그대로 있으면서 내부 상태가 바뀝니다.

객체는 시간에 따라 변경되는 상태를 포함하며, 행동을 통해 상태를 변경합니다. 따라서 객체는 가변 상태(mutable state)를 가진다고 말합니다.
객체는 두 객체를 동일하거나 다르다고 판단할 수 있는 프로퍼티를 가지며, 이를 식별자라고 합니다. 두 객체의 상태가 다르더라도 식별자가 같다면 두 객체를 같은 객체로 판단할 수 있다. 이처럼 식별자를 기반으로 객체가 같은지를 판단할 수 있는 성질을 **동일성**(identical)이라고 합니다.

객체의 특성을 정리하면 다음과 같습니다.

- 객체는 상태를 가지며 상태는 변경 가능하다.
- 상태를 변경하는 것은 객체의 행동이다. 행동의 결과는 상태에 의존하고, 행동의 순서가 결과에 영향을 미친다.
- 객체는 어떤 상태에 있더라도 유일하게 식별 가능하다.

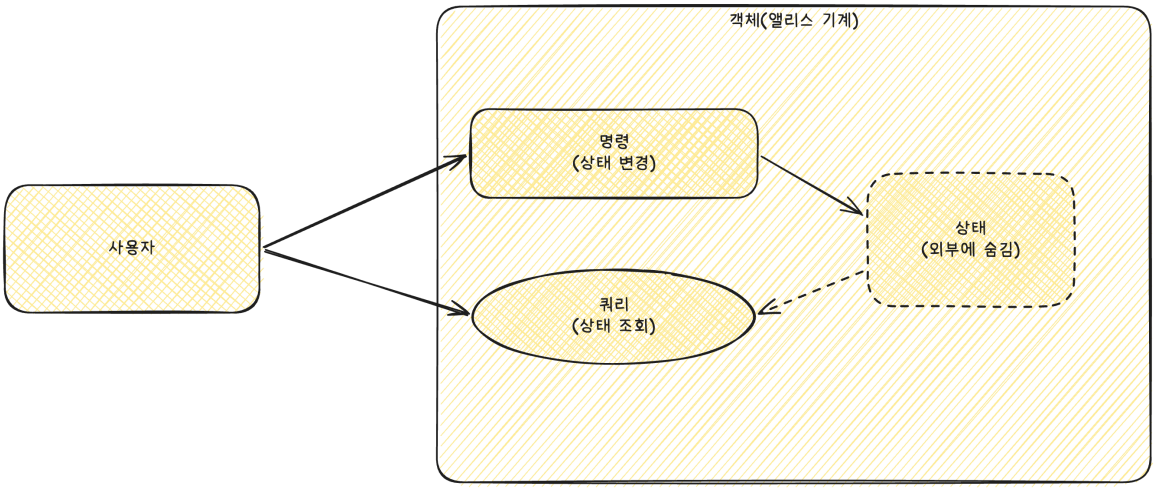
2.4. 기계로서의 객체

Bertrand Meyer가 제안한 비유는 객체를 금속 외피로 둘러싸인 블랙박스 기계로 보는 것입니다. 기계에는 두 종류의 버튼이 달려 있습니다.

- 사각형 버튼은 명령(command)으로, 눌러도 결과가 바로 보이지 않고 상태를 변경합니다.
- 둥근 버튼은 쿼리(query)로, 눌러서 상태를 조회하며 결과가 디스플레이에 나옵니다.
- 슬롯은 버튼을 누를 때 기계가 필요로 하는 추가 정보(인자)를 넣는 곳입니다.

> *"객체지향의 세계를 창조하는 개발자들의 주된 업무는 객체의 상태를 조회하고 객체의 상태를 변경하는 것이다."*

객체지향 개발자의 주된 일이 상태를 조회하고 변경하는 것이므로, 객체가 외부에 제공하는 행동은 대부분 Query와 Command입니다.



'객체 = 기계' 라는 비유를 통해, 아래의 객체지향 핵심 개념을 확인할 수 있습니다.

- **메시지**: 버튼을 누르는 것은 사용자이지만, 어떻게 동작할지는 기계가 스스로 결정합니다.
- **캡슐화**: 사용자는 버튼으로만 접근할 수 있고, 기계를 열어 내부 상태를 만지지 않습니다.
- **상태와 행동의 결합**: 기계는 버튼(행동)과 디스플레이(상태)를 함께 가집니다.
- **식별자**: 디스플레이에 똑같이 130이 찍힌 두 기계가 있어도 사람들은 별개의 기계로 인식합니다. 상태가 같아도 구별되는 정체가 있다는 뜻입니다.

2.5. 행동이 상태를 결정한다

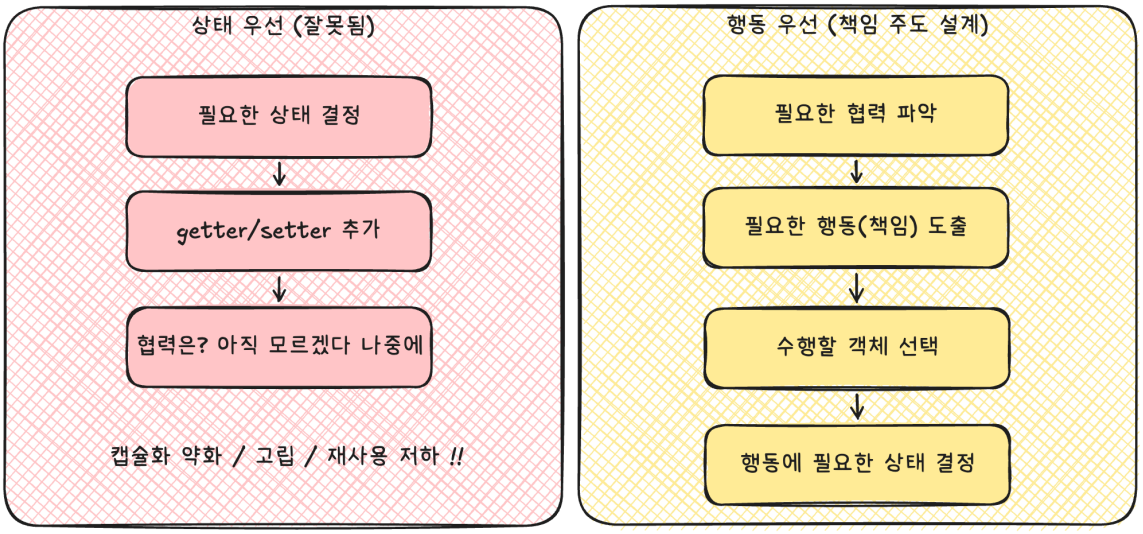
객체지향 입문자가 가장 쉽게 빠지는 함정은 상태부터 정하는 것입니다. 앨리스 객체를 설계할 때 '키와 위치가 필요하겠지'라며 필드를 먼저 만들고, 그다음에 그 필드를 읽고 바꾸는 메서드를 고민하는

식입니다. 원문은 이 방식이 세 가지 문제를 낳는다고 지적합니다.

- **캡슐화가 약해진다**: 상태에 초점을 맞추면 상태가 공용 인터페이스로 그대로 노출되기 쉽습니다.
- **객체가 협력자가 아닌 고립된 섬이 된다**: 객체는 협력하려고 존재하는데, 협력의 문맥을 벗어나 설계하게 됩니다.
- **재사용성이 낮아진다**: 재사용성은 다양한 협력에 참여할 수 있는 능력에서 나오는데, 상태 중심 객체는 그러기 어렵습니다.

> *"객체의 적합성을 결정하는 것은 상태가 아니라 객체의 행동이다."*

먼저 필요한 협력을 생각하고, 그 협력에 필요한 행동을 정하고, 그 행동을 수행할 객체를 고른 뒤, 마지막에 행동에 필요한 상태를 결정해야 합니다.



협력 안에서 객체의 행동은 곧 그 객체가 완수해야 할 책임입니다. 그래서 어떤 책임이 필요한가를 정하는 과정이 설계 전체를 이끌어야 하고, 이를 책임 주도 설계(RDD)라고 합니다. 행동을 먼저 정하면 행동에 필요한 정보만 상태로 남으므로 응집도 높고 재사용 가능한 객체가 만들어집니다. 1장의 `order.ship()`도 같은 맥락입니다. 상태를 꺼내 쓰는 대신 배송이라는 행동을 먼저 설계하면 상태는 그 행동 안으로 숨습니다.

2.5. 은유와 객체

객체지향은 현실을 그대로 복제하지 않습니다. 그럼에도 현실 세계의 개념을 소프트웨어 설계에 사용하는 이유는 현실 세계에서 익숙한 개념을 이용하면 소프트웨어 객체의 의미와 책임을 쉽게 이해할 수 있기 때문입니다.

현실의 상품은 스스로 가격을 계산하지 못하지만 소프트웨어의 상품은 계산합니다. 이것은 현실을 단순화한 추상화가 아니라 특성이 전혀 다른 존재라는 뜻입니다. 시뮬라가 이름부터 시뮬레이션을 내세웠더라도 현실을 그대로 베끼는 것은 오해입니다.

- **의인화(anthropomorphism)**: 현실에서 수동적인 존재가 소프트웨어에서는 능동적인 에이전트가 되는 것입니다. 현실의 전화기는 스스로 전화를 걸지 못하고 계좌는 스스로 이체하지 못하지만, 소프트웨어의 `Phone`과 `Account`는 합니다.
- **은유(metaphor)**: 한 대상의 의미 일부를 다른 대상으로 옮겨 이해하는 방식입니다. 예를 들어 소프트웨어 객체에 `Phone` 와 같은 현실의 이름을 붙이면 전화를 걸 수 있다는 사실을 쉽게 이해하고 기억할 수 있습니다.

은유가 중요한 이유는 **표현적 차이** 때문입니다. 사람들이 소프트웨어에 대해 생각하는 모습과 코드의 실제 표현 사이의 거리를 말합니다. 현실의 이름을 객체에 붙이면 이 거리가 줄어들어 구조를 쉽게 예측할 수 있고, 이해하기 쉽고 유지보수하기 쉬운 소프트웨어가 됩니다.

2.6. 이상한 나라를 창조하라

2장의 결론은 객체지향적으로 설계할 때 현실 세계에 얽매일 필요가 없다는 것입니다.

현실의 개념은 객체의 이름과 역할을 이해하기 위한 좋은 출발점이지만, 객체의 행동은 소프트웨어가 해결해야 하는 문제에 맞게 설계해야 합니다.

현실을 그대로 모방하는 것보다 중요한 것은 시스템에 필요한 책임을 적절한 객체에 부여하고, 객체들이 자율적으로 **협력**하도록 만드는 것이 중요하다고 강조하고 있습니다.

협력!!!!
협력 -> 행동(책임) -> 상태